ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
DEPARTMENT OF INFORMATICS

Computer Programming with Java Assignment

ATHANASIOS GOURDOMICHALIS

PART 1

Assignment Purpose

This assignment aims to develop an object-oriented system for representing, organizing, and performing basic processing on information related to movies and their reviews. We are required to design and implement class hierarchies, utilize core Java Collection frameworks, and develop searching, filtering, and ranking capabilities.

Context

An **online movie platform** wants to organize its movie catalog to better understand user preferences and improve the search and recommendation experience. Users can submit ratings and short reviews for movies they have watched.

In general, the platform aims to answer questions such as:

- Which are the highest-rated movies in each genre?
- How can it identify similar or related movies for recommendations?
- Which users have rated specific movies?

To support the above, the company requires the development of an object-oriented model that represents the core entities (**movies, users, reviews**), enables their insertion and retrieval, and provides searching, filtering, and ranking functionality.

Description

The system will represent information related to **movies, user reviews, and ratings**, to address queries such as:

- Which are the top movies by genre?
- What is the average rating of a movie?
- Which movies are linked to a specific movie based on genre?
- Which users have rated specific movies?

The data will be inserted programmatically (hardcoded), without using external files.

Design Requirements

The implementation must include at least the following core classes:

Movie

Attributes

- **Title**
- **Release year**
- **Genre / Genres** (List <String>)
- **Director**
- **Collection of reviews** (List<Review>)
- **Collection of related movies** (List<Movie>)

Review

Attributes

- **Associated user** (User)
- **Rating** (integer 1-10)

- `Optional comment`
- `Reference to the movie being evaluated`

`User`

Attributes

- `Username`
- `List of submitted reviews`

The implementation may be extended with additional attributes (e.g., **language, duration, actors**) or **methods**, provided this supports functionality for filtering, grouping, or displaying information.

Indicative Functionalities

- **Registration / Insertion** of a new movie, review, and user
- **Calculation of the average rating** per movie
- **Search for movies** based on title, genre, or rating
- **Display of top-rated movies per genre** (having at least **N** reviews and an average rating > **X**)
- **Movie association / recommendation** (same genre, common director, etc.)
- **Formatted display of information** using the toString() method

Polymorphism & Extensibility

The application's implementation must leverage polymorphism through class hierarchies or interfaces. Specifically:

Mandatory Requirements:

- **Abstract Review Hierarchy:** The Review class must be implemented as an abstract class or interface, with at least two concrete implementations (e.g., **BasicReview, VerifiedReview**) that differ in their implementation of the **`getWeightedRating()`** method.

  This method is defined in the abstract Review class and implemented by each subclass in a manner that reflects the "**weight**" or credibility of the review. It is used to calculate the average rating of a movie and enables the application of polymorphism. For example, **BasicReview** returns the plain rating (**`return rating`**), whereas **VerifiedReview** may return a weighted/boosted rating (e.g., **`return (int) Math.round(rating * 1.2)`**).

- **Extensible User Hierarchy:** The User class must be extensible (e.g., VerifiedUser, CriticUser) with differentiated behavior. That is, it should be inheritable, offer methods that can be overridden in subclasses, and allow behavior customization without altering the underlying base structure.

- **Common Display Interface:** The Movie, Review, and User classes must implement a common interface (e.g., Printable) containing a shared display method (`printDetails()`).

- **Custom Comparators:** Multiple Comparator implementations (e.g. **Comparator<Movie>**) must be used to sort entities based on various criteria (average rating, release year, etc.).

Assessment will focus on the understanding of **polymorphism** concepts and the ability to **design extensible class hierarchies**.

Code Draft (you may ignore or modify it)

```
// Movie.java
```

```
import java.util.*;
public class Movie implements Printable {
	private String title;
	private int year;
	private List<String> genres;
	private String director;
	private List<Review> reviews;
	private List<Movie> relatedMovies;

	public Movie(String title, int year, List genres, String director) {
		this.title = title;
		this.year = year;
		this.genres = new ArrayList<>(genres);
		this.director = director;
		this.reviews = new ArrayList<>();
		this.relatedMovies = new ArrayList<>();
	}

	public void addReview(Review r) {
		reviews.add(r);
	}

	public void addRelatedMovie(Movie m) {
		relatedMovies.add(m);
	}

	public double getAverageRating() {
		if (reviews.isEmpty()) return 0;
		int total = 0;
		for (Review r : reviews) {
			total += r.getWeightedRating();
		}
		return (double) total / reviews.size();
	}

	public void printDetails() {
		System.out.println("Title: " + title);
		System.out.println("Year: " + year);
		System.out.println("Genres: " + genres);
		System.out.println("Director: " + director);
		System.out.println("Average Rating: " + getAverageRating());
	}

	// Getters and other utility methods can be added here
}
```

```
// Review.java
public abstract class Review implements Printable {
	protected User user;
	protected int rating;
	protected String comment;
	protected Movie movie;

	public Review(User user, int rating, String comment, Movie movie) {
		this.user = user;
		this.rating = rating;
		this.comment = comment;
		this.movie = movie;
	}

	public abstract int getWeightedRating();

	public void printDetails() {
		System.out.println(user.getUsername() + " rated " + movie.getTitle()
	+ " with " + rating + "/10");
		if (comment != null && !comment.isEmpty())
			System.out.println("Comment: " + comment);
	}
}


// BasicReview.java
public class BasicReview extends Review {
	public BasicReview(User user, int rating, String comment, Movie movie) {
	super(user, rating, comment, movie);
}

public int getWeightedRating() {
	return rating;
	}
}


// User.java
import java.util.*;
public class User implements Printable {
	protected String username;
	protected List reviews;

	public User(String username) {
		this.username = username;
		this.reviews = new ArrayList<>();
	}
```

```
	public void addReview(Review r) {
		reviews.add(r);
	}

	public String getUsername() {
		return username;
	}

	public void printDetails() {
		System.out.println("User: " + username);
		System.out.println("Reviews submitted: " + reviews.size()); }
	}


	// Printable.java
	public interface Printable {
		void printDetails();
	}


	// Main.java
	import java.util.*;
	public class Main {
		public static void main(String[] args) {
			Movie m = new Movie("Inception", 2010, List.of("Sci-Fi",
		"Action"), "Christopher Nolan");
			User u = new User("alice");

			Review r = new BasicReview(u, 9, "Mind-blowing!", m);
			m.addReview(r);
			u.addReview(r);

			m.printDetails();
			System.out.println();
			u.printDetails();
			System.out.println();
			r.printDetails();
	}
}
```

Assignment Instructions

Submission Requirements:

1. **The assignment must be submitted as a Java project containing**:

   1.1. **Documented Source Code**: Well-commented source code.

   1.2. `README` **File**: A README file detailing the system architecture/design, features, and instructions for execution.

   1.3. A **demo** in `Main.java` showcasing representative use-case examples of the system's functionality.

2. **Implementation Guidelines:** During the implementation of our assignment, we have to adhere to these following rules:

   2.1. Entry Point: Name the class containing the main method **Main** and its corresponding file **Main.java**. Do not forget to include the personal details (names/IDs) of all team members as a comment at the top of this file.

   2.2. Package Structure: Do not group your classes **into Java packages** (keep all classes in the default package).

   2.3. Compilation & Execution: The submitted code must compile and run directly from the **command line** without relying on any Integrated Development Environment (IDE).